

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

OLLYDBG 软件破解

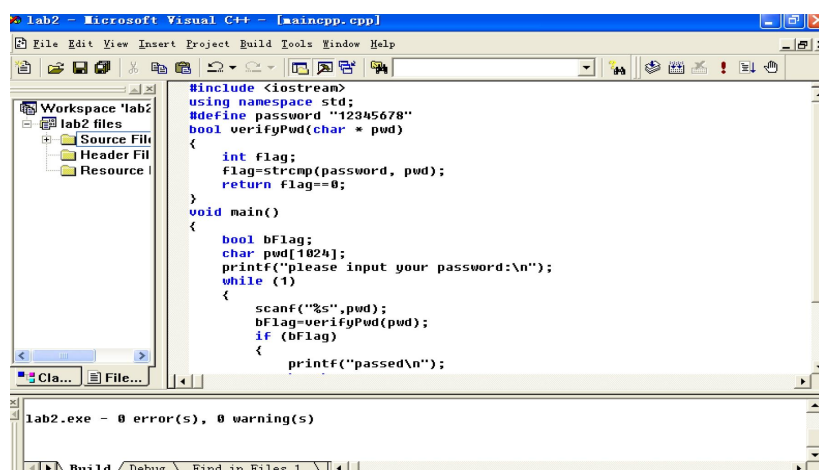
实验要求：

1. 请在 XP VC6 生成课本第三章软件破解的案例(DEBUG 模式, 示例 3-1)。进而, 使用 OlllyDBG 进行单步调试, 获取 verifyPwd 函数对应 flag==0 的汇编代码, 并对这些汇编代码进行解释。
2. 对生成的 DEBUG 程序进行破解, 复现课本上提供的两种破解方法。

实验过程：

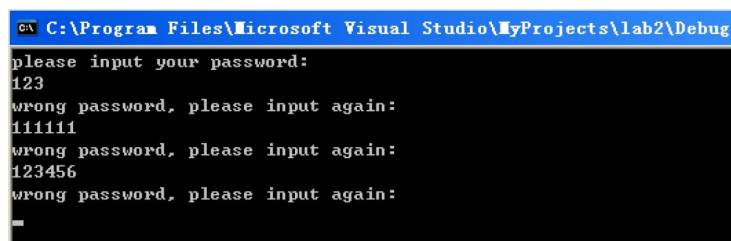
1. 测试与导入 OlllyDBG

打开 vmware 软件, 启动 XP 系统虚拟机, 打开 Microsoft Visual C++, 新建一个项目输入实验源码, 编译成可执行文件, 在 debug 文件夹中找到 exe 文件, 打开进行测试。



```
#include <iostream>
using namespace std;
#define password "12345678"
bool verifyPwd(char * pwd)
{
    int flag;
    flag=strcmp(password, pwd);
    return flag==0;
}
void main()
{
    bool bFlag;
    char pwd[1024];
    printf("please input your password:\n");
    while (1)
    {
        scanf("%s",pwd);
        bFlag=verifyPwd(pwd);
        if (bFlag)
        {
            printf("passed\n");
        }
    }
}
```

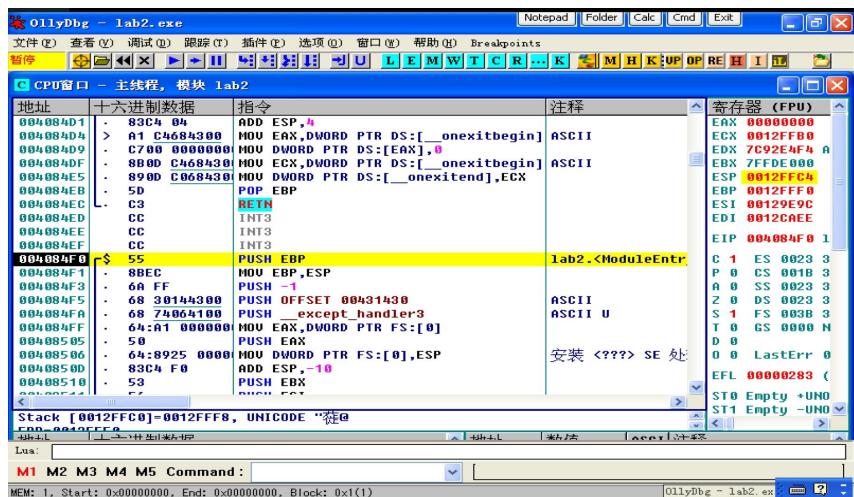
lab2.exe - 0 error(s), 0 warning(s)



```
C:\Program Files\Microsoft Visual Studio\MyProjects\lab2\Debug\
please input your password:
123
wrong password, please input again:
111111
wrong password, please input again:
123456
wrong password, please input again:
```

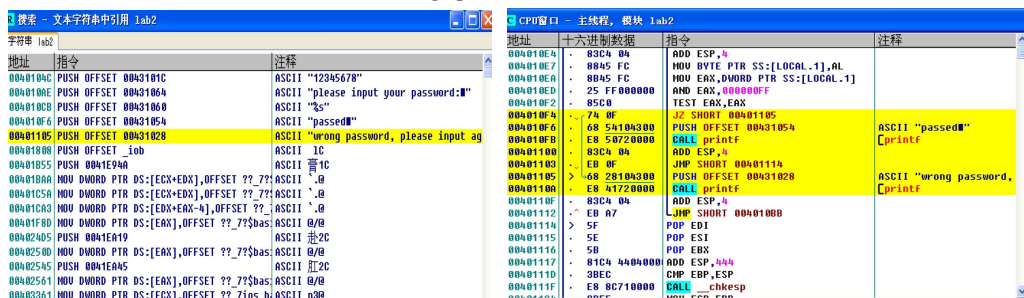
经过测试可见, 输入错误密码时会输出” wrong password, please input again”, 提示重新输入密码, 不能进入核心逻辑。这时运用 OlllyDBG 修改程序的汇编语言代码可以实现密码破解进入核心逻辑。

打开 OlllyDBG, 在 OlllyDBG 中选中 lab2.exe 文件打开进入调试。



2. verifyPwd 函数

这是关乎能否进入核心逻辑的关键函数。在 OllyDBG 中右键选中查找所有字符串进行关键字查询，找到类似于”wrong password”的关键字。



双击追踪，发现 verifyPwd 就是在这附近开始。选中它，右键跟随两次便来到了该函数所在位置。

地址	十六进制数据	指令	注释
0040102E	CC	INT3	
0040102F	CC	INT3	
00401030	> 55	PUSH EBP	lab2.verifyPwd(vd
00401031	8BEC	MOV EBP,ESP	
00401033	83EC 44	SUB ESP,44	
00401036	53	PUSH EBX	
00401037	56	PUSH ESI	
00401038	57	PUSH EDI	
00401039	8D7D BC	LEA EDI,[LOCAL.17]	
0040103C	B9 110000	MOV ECX,11	
00401041	B8 CCCCCC	MOV EAX,CCCCCCCC	UNICODE "???"
00401046	F3AB	REP STOS DWORD PTR ES:[EDI]	
00401048	8B45 08	MOV EAX,DWORD PTR SS:[ARG.1]	
00401048	50	PUSH EAX	
0040104C	68 1C104	PUSH OFFSET 0043101C	ASCII "12345678"
00401051	E8 CA710	CALL strcmp	[strcmp
00401056	83C4 08	ADD ESP,8	
00401059	8945 FC	MOV DWORD PTR SS:[LOCAL.1],EAX	
0040105C	33C0	XOR EAX,EAX	
0040105E	837D FC	CMP DWORD PTR SS:[LOCAL.1],0	
00401062	0F94C0	SETL AL	
00401065	5F	POP EDI	

分析核心代码（上图被框出部分）：

先是 MOV 指令，将输入的 password 的首地址移入 eax 中；PUSH 指令将 eax 入栈；接着第二个 PUSH，将待比对的正确密码（即 12345678）入栈；随后 CALL 指令，调用 strcmp 函数比对两个字符串是否相等，如果两个字符串相等，则 eax 的值为 0，如果不相等，则 eax 的值为 00000001；ADD 指令做栈平衡；然后，MOV 指令将 strcmp 的结果从 eax 中移送到局部变量 flag 中；接着 XOR 将 eax 寄存器的值清空；再用 CMP，对 flag 的值和 0 进行比较，如果相等的话就会设置 ZF 为 1，否则 ZF 为 0；

最后一行根据 ZF 的值设置 AL，如果 ZF 为 1，就设置 al 为 1，反之 al 被置 0。
接下来尝试破解。

3. 破解方法

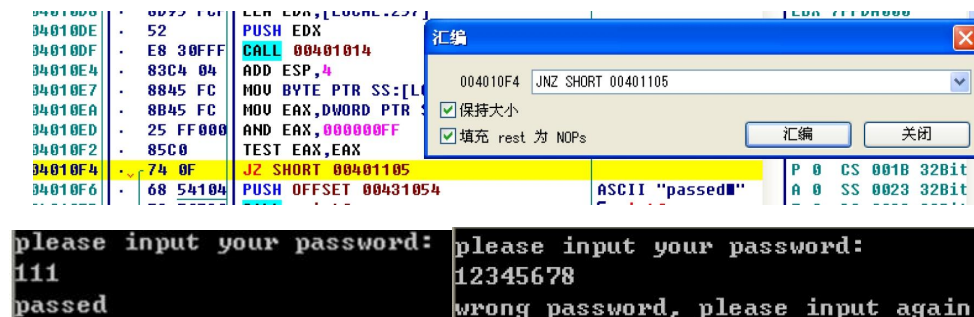
3.1>修改条件跳转指令

前面提到 verifyPwd 是关乎能否进入核心逻辑的关键函数，回到使用它判定的位置，发现是由一个 jz 判定语句决定是否跳出程序进入核心逻辑的。

004010F4	74 0F	JZ SHORT 00401105	
004010F6	68 54104300	PUSH OFFSET 00431054	ASCII "passed"
004010FB	E8 50720000	CALL printf	printf
00401100	83C4 04	ADD ESP,4	
00401103	EB 0F	JMP SHORT 00401114	
00401105	68 28104300	PUSH OFFSET 00431028	ASCII "wrong password,
0040110A	E8 41720000	CALL printf	printf

跟踪语句中提到的 00401105，发现其是用于密码错误的情况。可以分析出程序逻辑：如果输入密码正确，跳到” passed” 语句；输入密码错误，跳转到输出密码错误的语句。据此，我们可以修改 jz short 00401105 语句，既然这是跳转到输出错误语句，那么用 jnz 替换 jz 取反，改为 jnz short 00401105 语句，理论上便可以反方向跳转，在输入密码错误的时候” passed” 进入核心逻辑。

双击该语句实践修改，然后右键选择” 复制所有修改到可执行文件”，然后保存。打开修改后的 exe 文件。

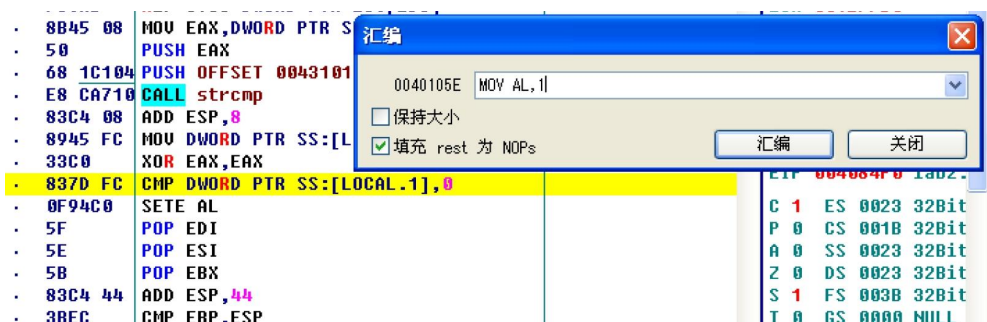


发现此时输入任何错误密码都能” passed”，进入核心逻辑；反而是输入正确密码即 12345678 输出” wrong password”。可见程序的密码判定逻辑被我们成功颠倒了。

3.2>修改 AL 赋值指令

通过分析 verifyPwd 函数，可以想到如果 flag==0 时返回值永远为 1，可以实现无论输入什么密码，都会” passed” 进入核心逻辑。

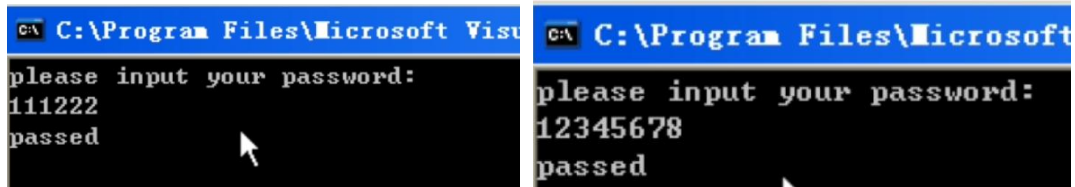
据此，找到 cmp 判断函数，将其删除，改为直接将 AL 赋值为 1 的语句。



接着，修改 SETE AL 语句。因为 AL 已经永 1 了，所以不需要该语句了，直接改为 nop 指令。

```
xor eax,eax  
mov al,1  
nop  
nop  
nop  
nop  
nop
```

最后是同样的保存操作。完成后打开修改后的 exe 文件测试一下。



发现此时输入任何密码都能” passed”，进入核心逻辑。程序的密码逻辑被我们成功破解了。

心得体会：

1. 学会了 ollydbg 的使用和调试，包括跟踪、查询、修改和保存编辑等常规操作。
2. 通过本实验，学习和了解了跟踪查找核心逻辑，并在一定程度上掌握了分析语句逻辑并作出修改进行破解的能力。
3. 对汇编语言有了更深入的理解。并且意识到分析能力是最基本也是最重要的能力。无论何时何事，面对逻辑都应有自己的分析和思考。